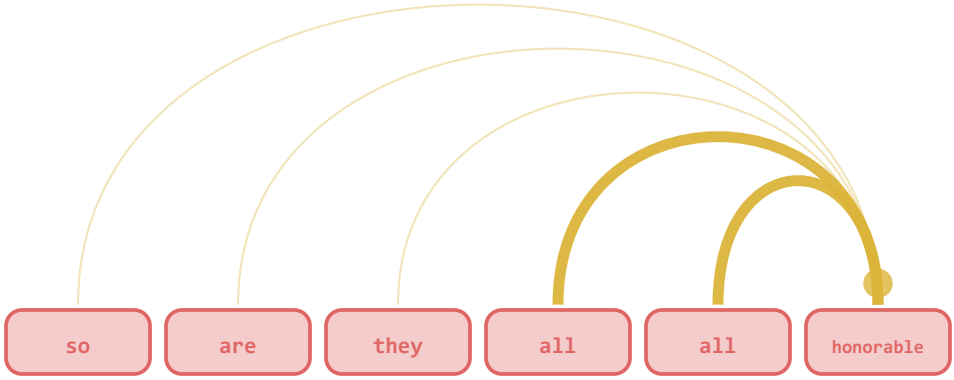


UNIT 01 · CHAPTER 8

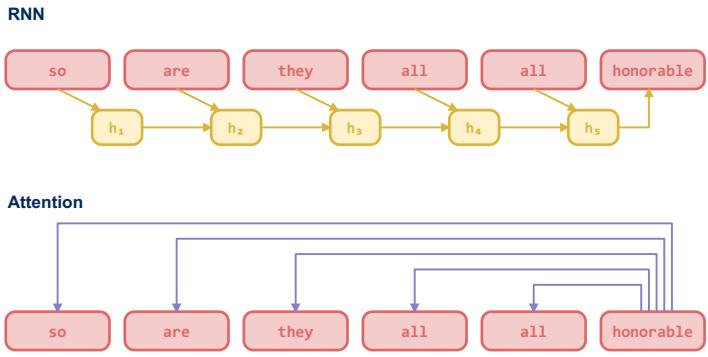
Self-Attention

CSE 10124 · Building ChatGPT

William Theisen · University of Notre Dame

Attention

Our RNN from Recurrent Neural Networks guessed "men" after "so are they all, all honorable", but the word that told it we meant more than one person had to survive three updates of h on the way. It also had to read the prompt one token at a time, since every h needed the one before it.



Pictured above is the alternative we ended on: **every token looks straight back at all the tokens before it** and pulls in whatever it needs, so nothing has to survive a chain of updates and no token waits for another.

This works because some parts of the context matter much more than others. For "honorable", whether we are talking about one person or several matters a lot, and "so" hardly matters at all. Letting a token decide how much each earlier token matters to it is called **attention**, and it needs two things: a way to score how much each token matters, and a way to use the scores.

Attention Scores

We already have a way to score how alike two tokens are. In Text Embeddings related words ended up with embeddings that have a large dot product, so a first try is to take the dot product of "honorable" with every token in the prompt.

We use the same speech and the same `16`-number random embeddings as in Recurrent Neural Networks, one row of `E` for each of the speech's `135` words:

```
import re, torch
text = open("shakespeare.txt").read()
speech = text[text.find("Friends, Romans"):]
speech = speech[:speech.find("come back to me.") + 16]
words = re.findall(r"[a-z']+", speech.lower())
vocab = sorted(set(words))
prompt = "so are they all all honorable".split()
ids = torch.tensor([vocab.index(w) for w in prompt])
torch.manual_seed(0)
E = torch.randn(len(vocab), 16)   # the embedding layer
emb = E[ids]                      # one row per token
torch.set_printoptions(precision=2, sci_mode=False)
scores = emb[-1] @ emb.T          # honorable vs. each
print(scores)
```

OUTPUT

```
tensor([ 1.06,  3.08, -6.58, -5.69, -5.69,  8.51])
```

Each of these `6` numbers is an **attention score**, how much "honorable" seems to care about each token in the prompt, "honorable" included. They can be any size and any sign. The biggest by far is `8.51`, "honorable" compared with itself.

Attention Weights

Scores that can be anything are hard to use. What we really want is a budget: "honorable" has a fixed amount of attention to spend and each token gets a share of it. That's exactly what the softmax from High Dimensional Data does, it turns any list of scores into positive numbers that add up to `1`:

```
weights = torch.softmax(scores, dim=0)
print(weights)
```

OUTPUT

```
tensor([ 1.06,  3.08, -6.58, -5.69, -5.69,  8.51])
tensor([0.00, 0.00, 0.00, 0.00, 0.00, 1.00])
```

These shares are the **attention weights**. Here "honorable" spends **all of its attention on itself**. Its own score (`8.51`) is `5.43` higher than the next best (`3.08` for "are"), and the softmax turns that gap into a factor of about `230` ($e^{5.43}$), so every other weight rounds to `0.00`.

Context

The weights say how much of each token "honorable" should take, so the natural thing to do with them is take it. The **context** for "honorable" is the average of the tokens' embeddings, each weighted by its attention weight. A row of weights times the matrix of embeddings is exactly that weighted average:

```
context = weights @ emb          # a weighted mix of rows
print(context[:4])
print(emb[-1, :4])               # honorable on its own
```

OUTPUT

```
tensor([ 1.06,  3.08, -6.58, -5.69, -5.69,  8.51])
tensor([0.00, 0.00, 0.00, 0.00, 0.00, 1.00])
tensor([-0.43,  0.54, -0.24,  0.21])
tensor([-0.44,  0.54, -0.24,  0.22])
```

The first **4** numbers of the context and of "honorable" on its own are **practically the same**. Our attention didn't look back at anything.

Unfortunately this will almost always happen. The dot product of a vector with itself is the sum of its squared numbers, which is hard for another, unrelated vector to beat, so a token scored by its raw embedding nearly always finds itself the most interesting token in the room. Comparing embeddings asks "which token is most like me?", and that's the wrong question. "Honorable" needs to ask something more like "which token tells me how many people we're talking about?".

Queries and Keys

The fix is to let what a token is looking for be different from what it has to offer. We multiply every embedding by two different weight matrices. W_Q turns each embedding into a **query** (what this token is looking for), and W_K turns each one into a **key** (what this token has). The attention score is then a query dot a key, not an embedding dot an embedding.

On the slides this is a dictionary lookup: we compare what we're looking for with every entry's headword and read the entries that match. Our W_Q and W_K are 16×4, so every query and key has 4 numbers (64 on the slides).

Queries and Keys in Code

In code we write `Wq` and `Wk`, since W is hard to type. Carrying on from the context code, the query of "honorable" now goes against every key:

```
torch.manual_seed(1)
Wq = torch.randn(16, 4) * 0.25    # Ω_Q
Wk = torch.randn(16, 4) * 0.25    # Ω_K
Q = emb @ Wq
K = emb @ Wk
scores = Q[-1] @ K.T              # honorable's query
print(scores)                     # vs. every key
print(torch.softmax(scores, dim=0))
```

OUTPUT

```
tensor([ 1.06,  3.08, -6.58, -5.69, -5.69,  8.51])
tensor([0.00, 0.00, 0.00, 0.00, 0.00, 1.00])
tensor([-0.43,  0.54, -0.24,  0.21])
tensor([-0.44,  0.54, -0.24,  0.22])
tensor([ 2.13,  0.96, -1.82, -2.56, -2.56,  0.58])
tensor([0.64, 0.20, 0.01, 0.01, 0.01, 0.14])
```

The last two lines are new. "Honorable" now spends `0.64` of its attention on "so", `0.20` on "are" and only `0.14` on itself. With random weights that means nothing (there's no reason to care about "so"), but it is no longer **stuck looking at itself**, since its query and its own key are different vectors. Learning the right W_Q and W_K is a job for training.

Values

Once "honorable" has decided which tokens to look at, it still has to take something from them. That something doesn't have to be the raw embedding either, so a third weight matrix W_V turns every embedding into a **value**, what the token actually hands over when it gets attention. Our W_V is 16×16, so a value has as many numbers as an embedding:




```python
Wv = torch.randn(16, 16) * 0.25   # Ω_V
V = emb @ Wv
weights = torch.softmax(Q[-1] @ K.T, dim=0)
context = weights @ V
print(context[:4])
```

OUTPUT

```
tensor([ 1.06,  3.08, -6.58, -5.69, -5.69,  8.51])
tensor([0.00, 0.00, 0.00, 0.00, 0.00, 1.00])
tensor([-0.43,  0.54, -0.24,  0.21])
tensor([-0.44,  0.54, -0.24,  0.22])
tensor([ 2.13,  0.96, -1.82, -2.56, -2.56,  0.58])
tensor([0.64, 0.20, 0.01, 0.01, 0.01, 0.14])
tensor([-0.91,  0.30, -0.62, -0.24])
```

The context is now the attention-weighted average of the values. Since queries, keys and values all come from the tokens of the same prompt, the prompt is attending to itself, which is why all this is called **self-attention**.

Every Token at Once

So far only "honorable" has looked back, but every token needs a context of its own. Stacking every query into Q and every key into K, one matrix product $Q\,K^T$ compares every query with every key at once:

```
import re, torch
text = open("shakespeare.txt").read()
speech = text[text.find("Friends, Romans"):]
speech = speech[:speech.find("come back to me.") + 16]
words = re.findall(r"[a-z']+", speech.lower())
vocab = sorted(set(words))
prompt = "so are they all all honorable".split()
ids = torch.tensor([vocab.index(w) for w in prompt])
torch.manual_seed(0)
E = torch.randn(len(vocab), 16)
torch.manual_seed(1)
Wq = torch.randn(16, 4) * 0.25
Wk = torch.randn(16, 4) * 0.25
torch.set_printoptions(precision=2, sci_mode=False)
emb = E[ids]
Q, K = emb @ Wq, emb @ Wk
scores = Q @ K.T                    # every query, every key
print(scores)
```

OUTPUT

```
tensor([[-0.06, -2.45,  0.58,  0.89,  0.89, -1.39],
        [-0.14,  2.20, -2.67, -0.96, -0.96,  1.38],
        [-1.72, -0.56,  0.01,  1.79,  1.79, -0.41],
        [-0.89,  0.48,  0.93,  0.88,  0.88,  0.02],
        [-0.89,  0.48,  0.93,  0.88,  0.88,  0.02],
        [ 2.13,  0.96, -1.82, -2.56, -2.56,  0.58]])
```

Row `i` holds token `i`'s scores against every key (the last row is "honorable" again), and all `36` come out of one multiplication with no loop over the tokens. That fixes **the RNN's second weakness**.

Looking Ahead

Notice that rows `4` and `5` of that grid are identical. Both are "all", with the same embedding and so the same query and key, and nothing here knows which one came first. Attention on its own doesn't know about word order at all (Transformer Blocks fixes that with positional encodings and RoPE).

Now look at the first row. "So" has scores against "are", "they" and every other token after it, so its context would include words that come later in the prompt.

That's a problem, because of what the model is for. When we train it, the job at "so" is to predict the next word, "are". If "so" can look at "are", the easiest way to lower the loss is to **copy the next word**, and a model that has learned to copy is useless when we generate text, since the next word doesn't exist yet.

So every token may look at itself and the tokens before it, and nothing after. The RNN got this for free, since h_t only ever saw the tokens up to t. With attention we have to enforce it ourselves.

Causal Masking

The trick is to change the scores before the softmax. Any score set to $-\infty$ gets a weight of exactly `0`, since $e^{-\infty}$ is `0`. `torch.tril` makes a lower triangle of ones (a token and everything before it), and `masked_fill` puts $-\infty$ everywhere else:

```
mask = torch.tril(torch.ones(6, 6))
scores = scores.masked_fill(mask == 0, float("-inf"))
print(scores)
print(torch.softmax(scores, dim=-1))
```

OUTPUT

```
tensor([[-0.06, -2.45,  0.58,  0.89,  0.89, -1.39],
        [-0.14,  2.20, -2.67, -0.96, -0.96,  1.38],
        [-1.72, -0.56,  0.01,  1.79,  1.79, -0.41],
        [-0.89,  0.48,  0.93,  0.88,  0.88,  0.02],
        [-0.89,  0.48,  0.93,  0.88,  0.88,  0.02],
        [ 2.13,  0.96, -1.82, -2.56, -2.56,  0.58]])
tensor([[-0.06,  -inf,  -inf,  -inf,  -inf,  -inf],
        [-0.14,  2.20,  -inf,  -inf,  -inf,  -inf],
        [-1.72, -0.56,  0.01,  -inf,  -inf,  -inf],
        [-0.89,  0.48,  0.93,  0.88,  -inf,  -inf],
```

This is a **causal mask**. "So" can only look at itself, so it puts all its attention there, and every row still adds up to `1` over the tokens it's allowed to see. The last row didn't change, since "honorable" was already last. The two "all" rows are different now, since **the second "all" can see the first**.

Scaling

There's one more problem, and it only shows up at real sizes. A dot product adds up one product per number in the query, so the more numbers, the bigger the scores. Here are `1000` dot products between random queries and keys of `4`, `64` and `1024` numbers:

```
import torch
torch.set_printoptions(precision=2, sci_mode=False)
torch.manual_seed(0)
for dk in [4, 64, 1024]:
    q = torch.randn(1000, dk)
    k = torch.randn(1000, dk)
    dots = (q * k).sum(dim=1)     # 1000 dot products
    print(dk, round(dots.std().item(), 1),
          round((dots / dk ** 0.5).std().item(), 2))
```

OUTPUT

```
4 2.0 1.0
64 8.1 1.01
1024 32.3 1.01
```

The spread of the scores (their standard deviation) grows with the square root of the size, `2.0`, `8.1` and `32.3`, and dividing by $\sqrt{d_k}$ brings it back to about `1`. That matters because of what the softmax does with big scores.

Scaling and the Softmax

Here is one 64-number query against six keys, with and without dividing by $\sqrt{64} = 8$:

```
q = torch.randn(64)              # one query
K = torch.randn(6, 64)           # six keys
scores = K @ q
print(scores)
print(torch.softmax(scores, dim=0))
print(torch.softmax(scores / 64 ** 0.5, dim=0))
```

OUTPUT

```
4 2.0 1.0
64 8.1 1.01
1024 32.3 1.01
tensor([ -5.88,  -0.36,  -9.04,   5.03,   7.72, -13.31])
tensor([0.00, 0.00, 0.00, 0.06, 0.94, 0.00])
tensor([0.07, 0.15, 0.05, 0.29, 0.41, 0.03])
```

Unscaled, the softmax gives one key **0.94** of the attention, and dividing by 8 spreads it out again. A softmax that is already all-or-nothing can hardly change, since a small nudge to a score barely moves a **0.00** or a **1.00**, so **training would get stuck**. Dividing every score by $\sqrt{d_k}$, where d_k is the size of a key, keeps them in a sensible range.

The Attention Equation

Put together, self-attention is one line, the equation from "Attention Is All You Need" (Vaswani and others, 2017), the paper that introduced the transformer. Scores from queries and keys, scaled, masked, a softmax across each row, and the weights times the values:

[8.1]

one row of context per token

$$\text{Context} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The mask isn't written in the equation, it's applied to the scores just before the softmax. For our prompt the shapes are:

matrix	ours	on the slides
$\mathcal{E}$	`<6, 16>`	`<7, 768>`
Q, K	`<6, 4>`	`<7, 64>`
V	`<6, 16>`	`<7, 768>`
attention weights	`<6, 6>`	`<7, 7>`
Context	`<6, 16>`	`<7, 768>`

Notice that the attention weights are always tokens × tokens, whatever the size of the embeddings, and the context always has the same shape as $\mathcal{E}$. Every token gets **one row of context, the same size as its embedding**.

Self-Attention in Code

Here is the whole thing in one cell, scaling and mask included. Dividing by $\sqrt{4} = 2$ makes the weights less extreme than on the masking page (`0.24` and `0.76` in the second row, not `0.09` and `0.91`):

```python
import re, math, torch
text = open("shakespeare.txt").read()
speech = text[text.find("Friends, Romans"):]
speech = speech[:speech.find("come back to me.") + 16]
words = re.findall(r"[a-z']+", speech.lower())
vocab = sorted(set(words))
prompt = "so are they all all honorable".split()
ids = torch.tensor([vocab.index(w) for w in prompt])
torch.manual_seed(0)
E = torch.randn(len(vocab), 16)
torch.manual_seed(1)
Wq = torch.randn(16, 4) * 0.25
Wk = torch.randn(16, 4) * 0.25
Wv = torch.randn(16, 16) * 0.25
torch.set_printoptions(precision=2, sci_mode=False)

emb = E[ids]
Q, K, V = emb @ Wq, emb @ Wk, emb @ Wv
scores = Q @ K.T / math.sqrt(4)
T = len(ids)
tril = torch.tril(torch.ones(T, T))
scores = scores.masked_fill(tril == 0, float("-inf"))
attn = torch.softmax(scores, dim=-1)
ctx = attn @ V
print(attn)
```

OUTPUT

```
tensor([[1.00, 0.00, 0.00, 0.00, 0.00, 0.00],
        [0.24, 0.76, 0.00, 0.00, 0.00, 0.00],
        [0.19, 0.35, 0.46, 0.00, 0.00, 0.00],
        [0.13, 0.25, 0.32, 0.31, 0.00, 0.00],
        [0.10, 0.19, 0.24, 0.23, 0.23, 0.00],
        [0.43, 0.24, 0.06, 0.04, 0.04, 0.20]])
```

Residual Connections

The context is what each token gathered from the others, but we don't want it to replace the token. With random weights "honorable" takes **43%** of its context from "so", and we'd hate for the model to forget which word it's actually sitting on. So we add the context to the embeddings instead:

[8.2] the context is added back to the embeddings

$$\mathcal{E}_{CA} = \mathcal{E} + \text{Context}$$

Adding the input back to the output like this is called a **residual connection**, and the results are **context-aware embeddings**, $\mathcal{E}_{CA}$.

Context-Aware Embeddings

Here are the first four numbers of the two "all"s before and after:

```
emb_ca = emb + ctx                # residual connection
print(emb[3, :4], emb[4, :4])     # the two "all"s
print(emb_ca[3, :4], emb_ca[4, :4])
```

OUTPUT

```
tensor([[1.00, 0.00, 0.00, 0.00, 0.00, 0.00],
        [0.24, 0.76, 0.00, 0.00, 0.00, 0.00],
        [0.19, 0.35, 0.46, 0.00, 0.00, 0.00],
        [0.13, 0.25, 0.32, 0.31, 0.00, 0.00],
        [0.10, 0.19, 0.24, 0.23, 0.23, 0.00],
        [0.43, 0.24, 0.06, 0.04, 0.04, 0.20]])
tensor([-0.16,  0.54,  0.21,  0.36]) tensor([-0.16,  0.54,  0.21,  0.36])
tensor([-0.63,  0.99, -0.30,  0.10]) tensor([-0.69,  1.24,  0.03, -0.05])
```

Going in, the two "all"s were the same vector. Coming out they are different, since **each one now carries its own context** (the second "all" saw the first one, the first didn't). This is what attention buys us. Every token's embedding has been updated with what it found looking back.

Producing Output

To predict the next word we do what the RNN did with h_t. We take the last row of $\mathcal{E}_{CA}$, multiply by W_y (16 × 135), add a bias b_y, and a softmax turns the 135 scores into probabilities:

[8.3] one probability per word in the vocabulary

$$\hat{y} = \mathrm{softmax}(\mathcal{E}_{CA}[-1]\, W_y + b_y)$$

The other rows aren't wasted. Row t of $\mathcal{E}_{CA}$ has seen tokens 1 to t, so it can predict token $t + 1$, and thanks to the mask it never saw the answer. During training **every row makes a prediction**, eight per window of eight words, all from one pass of attention.

That is the whole model: embeddings, one self-attention, a residual connection and an output layer. It has 4,839 weights, about a third of our RNN's 16,119, most of them in the embeddings and W_y.

irishQwen's attention (in `grouped_query_attention.py`) does the same steps with the same names, `scores`, `tril`, `attn` and `ctx`, just with much bigger matrices and a few extra pieces we'll meet later.

Training Samples

We train on the same samples as the RNN: every window of `8` words in the speech is an input, and the same window moved one word along is its target, `260` windows from the speech's `268` words. Here they are, along with every weight starting out small and random (the bias at zero, and the embedding `E` is learned too):

```
import re, math, torch
import torch.nn.functional as F

text = open("shakespeare.txt").read()
speech = text[text.find("Friends, Romans"):]
speech = speech[:speech.find("come back to me.") + 16]
words = re.findall(r"[a-z']+", speech.lower())
vocab = sorted(set(words))
ids = torch.tensor([vocab.index(w) for w in words])
starts = range(len(ids) - 8)
X = torch.stack([ids[i:i + 8] for i in starts])
Y = torch.stack([ids[i + 1:i + 9] for i in starts])

torch.manual_seed(0)
n = len(vocab)
E = torch.randn(n, 16) * 0.1
Wq = torch.randn(16, 4) * 0.1
Wk = torch.randn(16, 4) * 0.1
Wv = torch.randn(16, 16) * 0.1
Wy = torch.randn(16, n) * 0.1
by = torch.zeros(n)
params = [E, Wq, Wk, Wv, Wy, by]
for p in params:
    p.requires_grad_()
print(X.shape, Y.shape)
```

OUTPUT

```
torch.Size([260, 8]) torch.Size([260, 8])
```

Training Self-Attention

Attention runs on all `260` windows at once (hence `transpose(-2, -1)` instead of `.T`):

```
def attention(emb):
    T = emb.shape[-2]
    Q, K, V = emb @ Wq, emb @ Wk, emb @ Wv
    scores = Q @ K.transpose(-2, -1) / math.sqrt(4)
    tril = torch.tril(torch.ones(T, T))
    scores = scores.masked_fill(tril == 0, float("-inf"))
    attn = torch.softmax(scores, dim=-1)
    return emb + attn @ V, attn   # residual

opt = torch.optim.Adam(params, lr=0.01)
for step in range(301):
    emb_ca, attn = attention(E[X])
    logits = emb_ca @ Wy + by     # every position
    loss = F.cross_entropy(logits.reshape(-1, n),
                           Y.reshape(-1))
    opt.zero_grad()
    loss.backward()
    opt.step()
    if step % 100 == 0:
        print("step", step, "loss", round(loss.item(), 3))
```

OUTPUT

```
torch.Size([260, 8]) torch.Size([260, 8])
step 0 loss 4.901
step 100 loss 0.434
step 200 loss 0.297
step 300 loss 0.258
```

The loss starts at `4.901`, about where blind guessing among `135` words would be ($\ln 135 = 4.905$), and falls to `0.258`. Unlike the RNN there is **no loop over the eight positions**.

What It Predicts

Here are the trained model's two favorite next words for a few prompts, with their probabilities:

```
def predict(prompt):
    p = torch.tensor([vocab.index(w) for w in prompt])
    emb_ca, attn = attention(E[p])
    probs = torch.softmax(emb_ca[-1] @ Wy + by, dim=0)
    return probs, attn[-1]

with torch.no_grad():
    for prompt in ["honorable",
                   "and brutus is an honorable",
                   "so are they all all honorable",
                   "lend me your"]:
        p, i = predict(prompt.split())[0].topk(2)
        a, b = p.tolist()
        print(prompt.rjust(29), vocab[i[0]], round(a, 3),
              vocab[i[1]], round(b, 3))
```

OUTPUT

```
torch.Size([260, 8]) torch.Size([260, 8])
step 0 loss 4.901
step 100 loss 0.434
step 200 loss 0.297
step 300 loss 0.258
                    honorable man 0.801 men 0.198
   and brutus is an honorable man 0.999 men 0.0
so are they all all honorable men 0.999 i 0.0
                 lend me your ears 0.999 come 0.001
```

After "honorable" on its own the model hedges, **0.801** "man" and **0.198** "men", since the speech has both ("Brutus is an honorable man", "So are they all, all honorable men"). **Given the rest of the prompt it stops hedging**, "man" after "an honorable" and "men" after "all all honorable", both at **0.999**.

Where It Looks

To see how it stopped hedging we can print the last row of the attention weights, where "honorable" looked:

```
with torch.no_grad():
    for prompt in ["and brutus is an honorable",
                   "so are they all all honorable"]:
        attn = predict(prompt.split())[1]
        print([round(a, 2) for a in attn.tolist()])
```

OUTPUT

```
torch.Size([260, 8]) torch.Size([260, 8])
step 0 loss 4.901
step 100 loss 0.434
step 200 loss 0.297
step 300 loss 0.258
                    honorable man 0.801 men 0.198
   and brutus is an honorable man 0.999 men 0.0
so are they all all honorable men 0.999 i 0.0
                 lend me your ears 0.999 come 0.001
[0.04, 0.0, 0.01, 0.49, 0.47]
```

With "an" in the prompt, "honorable" spends `0.49` of its attention on "an". With "all all" it spends `0.36` on each "all". Nobody told the model that "an" means one person and "all" means several, **it worked out where to look** because looking there lowered the loss.

Attention During Training

Pictured below is that same row as training goes on, for both prompts:

	"and brutus is an honorable" → man					"so are they all all honorable" → men					
	and	brutus	is	an	honorable	so	are	they	all	all	honorable
untrained	0.20	0.20	0.20	0.20	0.20	0.17	0.17	0.17	0.17	0.17	0.17
step 50	0.16	0.01	0.13	0.38	0.32	0.01	0.08	0.02	0.14	0.14	0.62
step 100	0.10	0.01	0.04	0.69	0.16	0.00	0.00	0.02	0.40	0.40	0.18
trained	0.04	0.00	0.01	0.49	0.47	0.00	0.00	0.00	0.36	0.36	0.27

Untrained, W_Q and W_K are so small that every token gets an equal share, and the weights move to "an" and "all" within the first `100` steps.

Generating Text

To generate text we do what we did with the RNN. Predict the next word, append it to the prompt, run attention again on the longer prompt, and repeat. Each new word is predicted from the words before it, including the ones the model wrote itself, which is called **auto-regressive** generation. Starting from "friends romans" and always taking the most likely word (the model only ever saw windows of `8`, so we give it the last `8`):

```
text = ["friends", "romans"]
with torch.no_grad():
    for _ in range(20):
        probs = predict(text[-8:])[0]   # last 8 words
        text.append(vocab[probs.argmax().item()])
print(" ".join(text[:11]))
print(" ".join(text[11:]))
```

OUTPUT

```
torch.Size([260, 8]) torch.Size([260, 8])
step 0 loss 4.901
step 100 loss 0.434
step 200 loss 0.297
step 300 loss 0.258
                    honorable man 0.801 men 0.198
   and brutus is an honorable man 0.999 men 0.0
so are they all all honorable men 0.999 i 0.0
                 lend me your ears 0.999 come 0.001
[0.04, 0.0, 0.01, 0.49, 0.47]
```

That's the speech, nearly. Antony says "I come to bury Caesar", and our model **slips in an extra "i"**, then recovers. With `4,839` weights and `260` windows it has mostly memorized the speech, which is all we asked of it.

One Question per Token

Look back at where "honorable" looked in "so are they all, all honorable": `0.36` on each "all", `0.27` on itself and nothing on "they". That was enough here, since "all" answers the one question the model needed answered, one person or several.

Unfortunately a token gets just one row of attention weights, and it adds up to `1`. If "honorable" also needed to know who is being described, every bit of attention it spent on "they" would have to come away from "all". **One row of weights can only strike one balance** between the questions a token has.

Our W_Q, W_K and W_V are a single **attention head**, one set of queries, keys and values, one question per token. Real transformers give every token several heads, each with its own W_Q, W_K and W_V and its own row of weights, and that's where the next chapter, Multi-Head Attention, picks up.